Attention

1. from [6]
    1. Attention
        1. Soft attention mechanisms use learned weights:
            1. Channel-wise attention: SE Net. Related: see Squeeze Excitation Block in Image_Recognition.
                1. Weights different channels by multiplying with channel-specific weights
                2. winning entry of ImageNet classification task (2017)
            2. Spatial attention: Spatial Transformer Re-maps pixels and crops images.
                1. Transforms each channel of spatial pixel in the same way.
                2. Learns a 6 dof affine transformation in 2D.
        2. Modular way of incorporating Attention:
            1. STN works in a similar way to SE Net.
            2. Produces an output block of the same size as input.
                1. Insert between any pair of convolution layers (similar to SE Net)
                2. No change to existing architecture other than attention
        3. What a Spatial Attention can do
            1. Can rotate, translate, reflect, or scale images
            2. Transformation is image-specific
            3. Magnification and translation automatically leads to cropping and attention!
                1. Attention is a special case of this more general approach
                2. Always learned spatial invariance and is therefore more discriminative than max-pooling
        4. Components of Spatial Transformer Network (STN):
            1. Localization network
                1. A regression network with 6 outputs, six image-specific values that are reshaped into a 3×2 matrix $\Rightarrow$ Defines the mapping between old and new image coordinates.
                2. Grid generator: Transforms the 2D pixel coordinates (28×28 of them) with the generated matrix to map to (possibly fractional) coordinates.
                3. Sampler: Creates new 28×28 pixel values based on mapping.
                4. Related: see Bilinear Interpolation in Bilinear_Interpolation.
                5. Six entries of A produced by localization network in image-specific manner.
                6. Localization network can be CNN or even FCN.
            2. Grid generator
                1. We need pixel values for every coordinate pair $i, j \in 28 \times 28$ in transformed image.
                2. Easier to use A to reverse map target coordinates i, j to coordinates x_{ij}, y_{ij} in original image.
                3. Scaling factors inverses of eigenvalues of S in $A = US$.
            3. Interpreting generated coordinates
                1. The pixel value i, j in the target image is equal to the pixel value $x_{ij}, y_{ij} = [i, j, 1]A$ in source image.
                2. Solution: Use interpolation from four surrounding pixels x_{ij}, y_{ij} in source image.
                    1. Feature: results in (sub)-differential mapping, which is good for backpropagation.
        5. from Kevin Zakka:
            1. Affine transformation:
                1. In 2D, transformation matrix M (without translation) is a 2×2 matrix. $M = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$.
                2. Scaling $M = \begin{bmatrix} p & 0 \\ 0 & q \end{bmatrix}$. A special case is when $p = q = s$ which is called isotropic scaling. If $s > 1$ we are enlarging an image while shrinking would correspond to $s < 1$.
                3. Rotation $M = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$.
                4. Shearing $a = d = 1$, $b = m$, $c = n$, $M = \begin{bmatrix} 1 & m \\ n & 1 \end{bmatrix}$.
                5. we have defined 3 basic linear transformations:
                    1. scaling: scales the x and y direction by a scalar.
                    2. shearing: offsets the x by a number proportional to y and x by a number proportional to x.

               3. rotating: rotates the points around the origin by an angle θ.
            6. we can collapse sequential linear transformations into a single transformation matrix. Say we want to transform K by a shear, a scale, and a rotation. Given that these transformations can be represented by the matrices H, S, and R, and respecting the order of transformations, we can write down this operation as $K' = R[S(HK)]$. This can be reduced to $K' = MK$ where $M = RSH$. Be mindful of the order since matrix multiplication is **not** commutative.
            7. To represent a translation we have to place 2 new parameters e and f and we represent our coordinates using homogeneous coordinates.
            8. $M = \begin{bmatrix} a & b & e \\ c & d & f \\ 0 & 0 & 1 \end{bmatrix}$. Hence, we can generalize our results and represent our 4 affine transformations (all linear transformations are affine) by the 6 dof matrix $M = \begin{bmatrix} a & b & e \\ c & d & f \end{bmatrix}$.
2. Attention module:
   1. Channel-wise attention
      1. Squeeze-and-Excitation (SE block), Related: see Squeeze Excitation Block in Image_Recognition.
   2. Spatial-wise attention:
      1. Spatial Attention Module (SAM)
         1. CBAM
            1. Given an intermediate feature map F : (C, H, W) as input, CBAM sequentially infers a 1D channel attention map M_c: (C, 1, 1) and a 2D spatial attention map M_s: (1, H, W). The overall process can be summarized as $F' = M_c(F) \odot F$, and $F'' = M_s(F') \odot F'$.
            2. where $\odot$ denotes element-wise multiplication. The attention values are broadcasted accordingly during multiplication: channel attention values are broadcasted along the spatial dimension, and vice versa.
3. [1], demonstrates a special case of attention:
   1. Self-attention is a seq2seq operation: a sequence of vectors goes in, and a sequence of vectors comes out. Let's call the input vectors $\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_t$ and the corresponding output vectors $\vec{y}_1, \vec{y}_2, \ldots, \vec{y}_t$. The vectors all have dimension k.
   2. To produce output vector $\vec{y}_i$, the self attention operation simply takes a **weighted average over all the input vectors** $\vec{y}_i = \sum_j w_{ij} \vec{x}_j$.
   3. where j indexes over the whole sequence and the weights sum to one over all j. The weight w_{ij} is not a parameter, as in a normal neural net, but is is *derived* from a function over $\vec{x}_i$ and $\vec{x}_j$.
   4. The simplest option for this function is the dot product:
      1. $w'_{ij} = \vec{x}_i^T \vec{x}_j$.
      2. Note: $\vec{x}_i$ is the input vector at the same position as the current output vector $\vec{y}_i$. For the next output vector, we get an entirely new series of dot products, and a different weighted sum.
   5. The dot product gives us a value anywhere between negative and positive infinity, so we apply a softmax to map the values to $[0, 1]$ and ensure that they sum to 1 over the whole sequence:
      1. $w_{ij} = \frac{\exp w'_{ij}}{\sum_j \exp w'_{ij}}$.
   6. And that's the basic operation of self-attention.
4. A simple description of self-attention differs with fully-connected layers [2]:
   1. Ignoring details like normalization, biases, etc, fully connected networks are fixed (in position to the features) weights:
      1. $f(x) = (Wx)$ where W is learned in training, and fixed in inference.
   2. Self-attention layers are dynamic, changing the weight as it goes:
      1. $\mathrm{attn}(x) = (Wx)$
      2. f(x) = (attn(x) * x)
5. How attention works, [3]:
   1. Special case of attention:
      1. We have a query vector $\vec{q}$ (e.g. the decoder state)
      2. We have input vectors $H = \left[\vec{h}_1, \ldots, \vec{h}_L\right]^T$ (e.g. one per source word, encoder state)
      3. We compute affinity scores $s_1, \ldots s_L$ by "comparing" q and H. (H appeared here)
      4. We convert these scores to probabilities:
         1. $\vec{p} = \mathrm{softmax}(\vec{s})$
      5. We use this to output a representation as a weighted average:
         1. $c = H^T p = \sum_{i=1}^{L} p_i \vec{h}_i$ (and here)
   2. Affinity scores (attention scores):

- Several ways of "comparing" a query $\vec{q}$ and an input "key" vector $\vec{h}_i$:
  1. Additive attention (Bahdanau et al. 2015):$s_i = u^T \tanh(A\vec{h}_i + B\vec{q})$
  2. Bilinear attention (Luong et al., 2015): $s_i = \vec{q}^T U \vec{h}_i$
  3. Dot product attention (Luong et al. 2015): particular case of bilinear attention; queries and keys must have the same size, $s_i = \vec{q}^T \vec{h}_i$, $O(n^2)$ complexity.
- The last two are easy to batch for multiple queries and multiple keys.

3. The input vectors $H = \left[\vec{h}_1, \ldots, \vec{h}_L\right]$ appear in two places:
   1. They are used as keys to "compare" them with the query vector $\vec{q}$ to obtain the affinity scores.
   2. They are used as values to form the weighted average $\vec{c} = H^T \vec{p}$.
4. To be fully general, they don't have to be the same -- we can have:
   1. A key matrix $K = \left[\vec{k}_1, \ldots, \vec{k}_L\right]^T \in \mathrm{R}^{L \times d_K}$
   2. A value matrix $V = [\vec{v}_1 \ldots \vec{v}_L]^T \in \mathrm{R}^{L \times d_V}$
5. We have a more general version of attention mechanism:
   1. We have a query vector $\vec{q}$ (e.g. the decoder state)
   2. We have key vectors $K = \left[\vec{k}_1 \ldots \vec{k}_L\right]^T \in \mathrm{R}^{L \times d_K}$
   3. and value vectors $V = [\vec{v}_1 \ldots \vec{v}_L]^T \in \mathrm{R}^{L \times d_V}$, (one of each per source word)
   4. We compute query-key affinity scores $s_1, \ldots, s_L$ comparing q and K.
   5. We convert these scores to probabilities: $p = \mathrm{softmax}(\vec{s})$
   6. We output a weighted average of the values: $c = V^T \vec{p} = \sum_{i=1}^{L} p_i \vec{v}_i \in \mathrm{R}^{d_V}$
6. Self-attention for a sequence of length L:
   1. Query vectors
   2. Key vectors
   3. Value vectors

6. Attention is a set operator
7. Q, K, V in Attention:
   1. Query: This is what **pays the attention**.
   2. Values: These are **paid attention to**.
   3. Keys: These help queries figure out **how much attention to pay** to each of the values.
   4. Attention Weights: How much attention to pay.
8. Multi-headed self-attention
   1. Split input into k parts.
   2. Pass the $j-$th part of each input into the $j-$th attention head.
   3. Concatenate each of the k outputs.
9. Why go through the trouble?
   1. Each head could find a different kind of relation between the tokens.
      1. Subject-verb, subject-object, verb-modifier, dependency, etc.
10. To calculate attention weights for input h_i, you would use query q_i, and all keys.
11. The energy function is scaled to allow for numerical stability.
12. The attention module tries to calculate the "shift" in meaning of a token given all other tokens in the batch.
13. Transformer decoders can only be parallelized during training.
14. Multiheaded attention helps transformers find different kinds of relations between the tokens.
15. Self attention and cross attention:
    1. Key and value from the same source.
    2. If query is from the same source as key and value -- self attention.
    3. If query is form a different source -- cross attention.
    4. Cross-attention highlights the matching/alignment between a word in the output sequence and its corresponding word or words in the input sequence.
    5. Unlike the self-attention mechanism in the decoder layer, the cross-attention layer **does not require a mask**.
    6. In fact, applying a mask would be counter-productive, as the query maximizes the prediction of the next word in the output sequence by **attending to all tokens in the input sequence**. [8], [9]

References:

1. https://peterbloem.nl/blog/transformers
2. self attention vs fully connected layer. stackoverflow.
3. Lecture 10, A. Martins, C. Zerva, M. Figueiredo, IST Lisboa.
4. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention, Angelos Karthapoulos.
5. Effective Approaches to Attention-based Neural Machine Translation. Minh-Thang Luong.
6. Spatial Attention layer in Computer Vision: The Spatial Transformer. Youtube. Charu Aggarwal.
7. Cordonnier et. al. On the Relationship between self-Attention and Convolutional layers. ICLR 2020.
8. Lecture 19, Transformers and GNNs, CMU 11-785 F22.
9. Xavier Bresson, Lecture 7, Attention Neural Networks.